

Voice Action Gate

AI can understand your intent. It cannot invent your authorization.

Version	1.0
Status	MVP / Hackathon
Product Type	AI Voice Agent Security & Authorization Layer
Primary Interface	Real-time Voice
Target Users	Developers, DevOps/SRE teams, IT administrators, business operators

1. Product Overview

1.1 Product Name

Voice Action Gate

1.2 Tagline

“AI can understand your intent. It cannot invent your authorization.”

1.3 Executive Summary

Voice Action Gate is a security and authorization layer designed for AI voice agents that can execute real-world actions through tools and APIs.

Modern AI agents can understand natural language and call tools autonomously. However, language models frequently infer missing information from context.

For low-risk tasks, this is useful. For sensitive or irreversible actions, it can be dangerous.

Voice Action Gate solves this problem by placing a verification layer between the AI agent and the action execution system.

Before executing a sensitive action, Voice Action Gate verifies:

- What action the user requested
- Which parameters are required
- Which parameters were explicitly spoken
- Which values were inferred
- Whether the request is ambiguous
- The risk level of the action
- Whether explicit confirmation is required

- Whether all authorization requirements have been satisfied

Only when the request passes the Action Gate can the underlying tool execute.

2. Problem Statement

AI agents are increasingly capable of performing actions rather than simply generating text.

For example:

“Deploy the latest version.”

An AI agent might infer:

- Which application
- Which version
- Which environment
- Whether deployment should happen immediately

Similarly:

“Delete the old database.”

The AI may determine what “old” means based on available context.

This creates a critical security problem:

AI inference $\neq$ user authorization.

A user may have intended one action while the AI executes another.

Existing AI agents often optimize for task completion. Voice Action Gate optimizes for:

“Safe, explicit, traceable execution.”

3. Product Vision

Create a universal **authorization gateway for autonomous AI agents**.

Any AI agent capable of calling tools should be able to route sensitive actions through Voice Action Gate.

The system should answer:

“Did the user actually say enough for us to safely perform this action?”

4. Goals

Primary Goals

- Prevent execution of sensitive actions using inferred parameters.
- Verify that required parameters were explicitly provided.

- Detect ambiguous or incomplete requests.
- Require confirmation for high-risk actions.
- Provide real-time voice clarification.
- Maintain a complete audit trail.
- Demonstrate the system across multiple action types.
- Integrate with an AI voice agent architecture.

Secondary Goals

- Make the authorization engine reusable.
- Provide a visual dashboard for monitoring actions.
- Allow developers to define custom action schemas.
- Provide explainable reasons for blocked actions.

5. Non-Goals

The MVP will NOT attempt to:

- Replace enterprise IAM systems.
- Provide real financial transactions.
- Execute real destructive production operations.
- Become a general-purpose cybersecurity platform.
- Guarantee perfect semantic understanding.
- Replace human security policies.

All destructive/financial demonstrations should use simulated environments.

6. Target Users

6.1 DevOps Engineers

Use AI agents to:

- Deploy applications
- Restart services
- Modify infrastructure
- Manage containers
- Perform database operations

6.2 IT Administrators

Use AI agents to:

- Create accounts

- Disable accounts
- Change permissions
- Modify configurations

6.3 Developers

Use AI agents to:

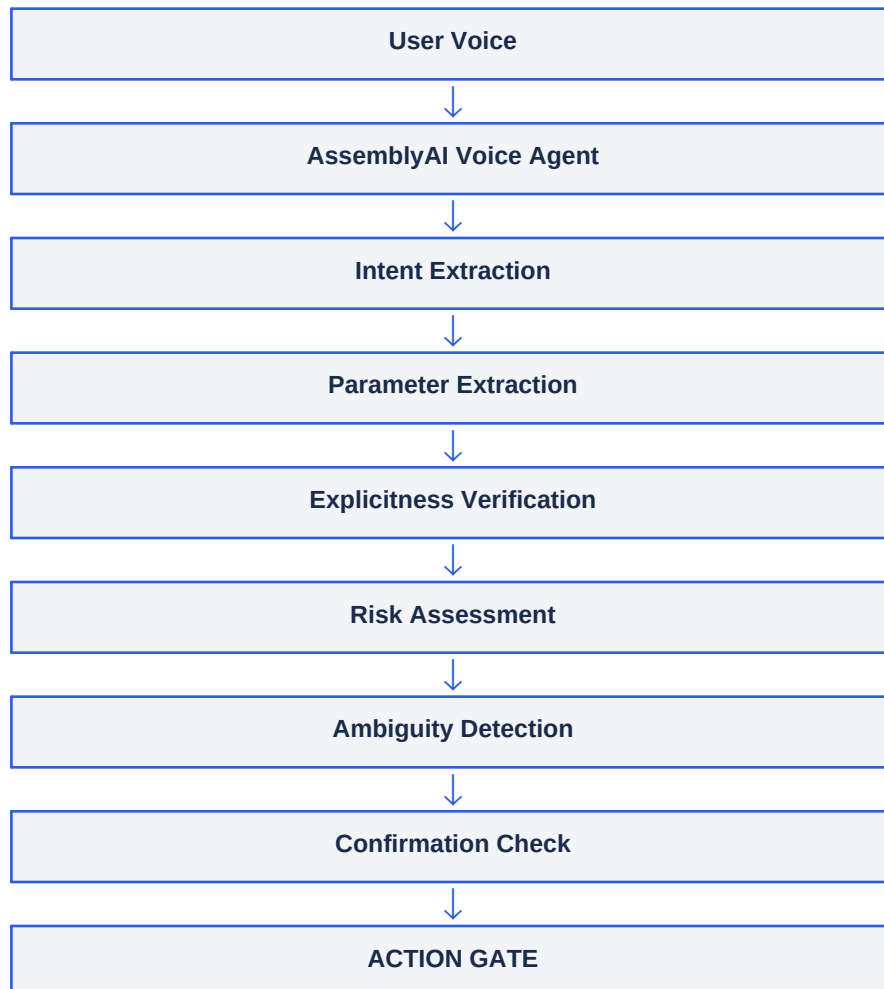
- Execute development tools
- Manage repositories
- Run deployments
- Modify databases

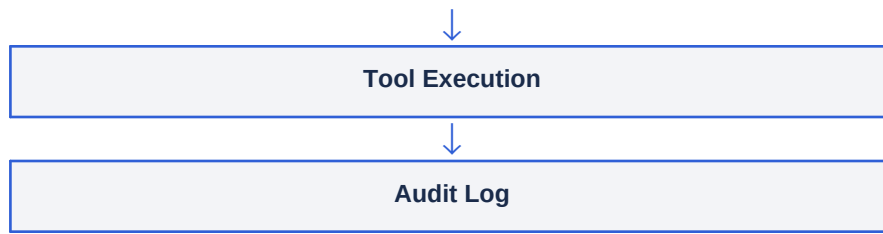
6.4 Enterprise Operators

Use voice agents to perform business workflows while maintaining authorization and auditability.

7. Core Product Concept

Voice Action Gate introduces a controlled pipeline:





The critical component is the **Action Gate**.

8. Core Principle

Explicit Authorization Principle

A parameter required for a sensitive action must be explicitly supplied by the user.

The system must distinguish between:

Explicit

User:

“Deploy version 4.8.2 to production.”

```
version = 4.8.2
source = user_explicit

environment = production
source = user_explicit
```

Inferred

User:

“Deploy the latest version.”

System knows:

```
latest_version = 4.8.2
```

But:

```
source = system_inference
```

For a high-risk action:

```
INFERENCE ≠ AUTHORIZATION
```

The action must be blocked until the user explicitly provides the missing value.

9. Risk Classification

Every action must have a risk level.

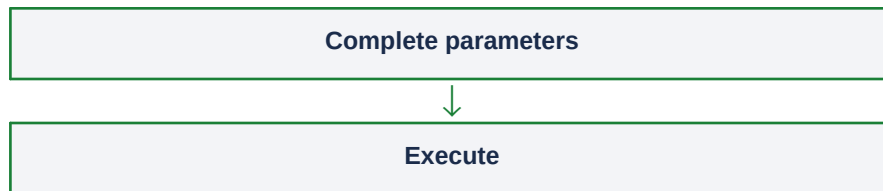
Low Risk

LOW RISK

Examples:

- Search information
- View analytics
- Generate report
- Read logs

Behavior:



No confirmation required.

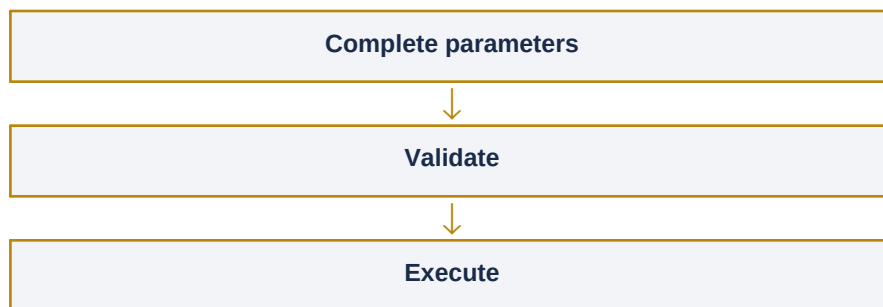
Medium Risk

MEDIUM RISK

Examples:

- Create ticket
- Restart development service
- Send internal notification
- Modify non-critical data

Behavior:



High Risk

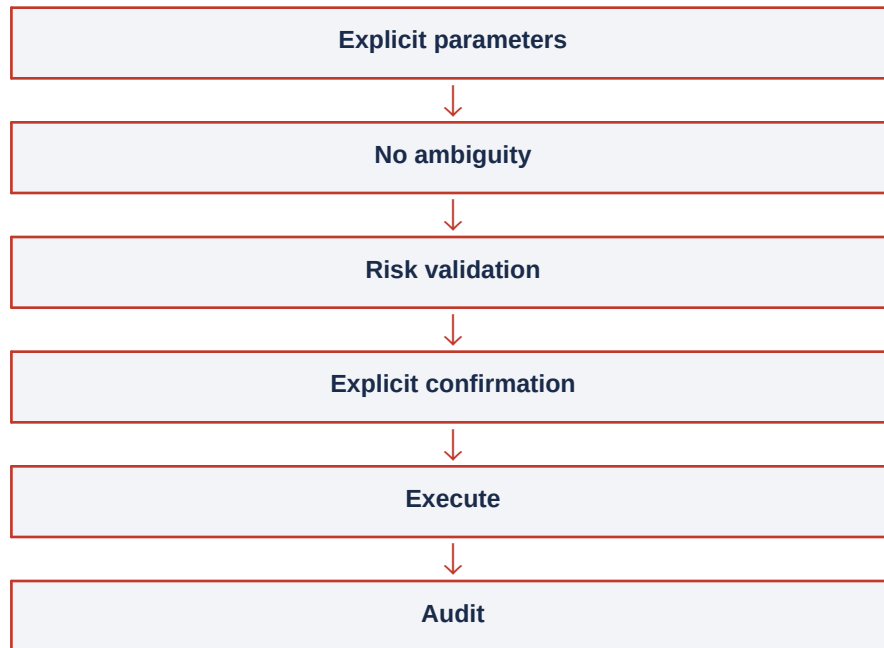
HIGH RISK

Examples:

- Production deployment
- Database deletion

- Permission modification
- Account disabling
- Financial transfer

Behavior:



10. MVP Features

Feature 1 — Real-Time Voice Interaction

The user communicates naturally with the agent.

Example:

"Deploy version 4.8.2 to production."

The voice agent processes the request in real time.

Requirements

- Speech input
- Speech-to-text
- Natural language understanding
- Voice response
- Interruption handling
- Low-latency conversation

11. Feature 2 — Intent Detection

The system identifies the requested action.

Example:

```
User:
"Deploy version 4.8.2 to production."
```

Extract:

```
{
  "intent": "DEPLOY_APPLICATION"
}
```

12. Feature 3 — Parameter Extraction

The system extracts action parameters.

Example:

```
{
  "action": "DEPLOY_APPLICATION",
  "parameters": {
    "application": null,
    "version": "4.8.2",
    "environment": "production"
  }
}
```

13. Feature 4 — Parameter Provenance

Every extracted parameter must contain its source.

Example:

```
{
  "version": {
    "value": "4.8.2",
    "source": "user_explicit"
  },
  "environment": {
    "value": "production",
    "source": "user_explicit"
  }
}
```

Possible source types:

```
USER_EXPLICIT
SYSTEM_CONTEXT
AI_INFERENCE
DEFAULT_VALUE
UNKNOWN
```

For high-risk actions, only:

```
USER_EXPLICIT
```

should satisfy required authorization.

14. Feature 5 — Action Schema

Every executable action has a defined schema.

Example:

```
{
  "action": "DELETE_DATABASE",
  "risk": "HIGH",
  "required_parameters": [
    "database",
    "environment"
  ],
  "requires_confirmation": true
}
```

This allows Voice Action Gate to work independently of any particular tool.

15. Feature 6 — Missing Parameter Detection

If a required parameter is missing, execution is blocked.

Example:

User:

"Delete the customer database."

System:

```
Action:
DELETE_DATABASE

Database:
customer_db ■

Environment:
MISSING ■

Confirmation:
MISSING ■
```

Result:

```
BLOCKED
```

Agent:

"Which environment is the customer database in?"

16. Feature 7 — Ambiguity Detection

The system must detect uncertain values.

Example:

User:

"Delete the old database."

Potential databases:

```
customer_db
customer_db_backup
customer_db_archive
```

The agent must not choose one automatically. Instead:

"I found multiple possible databases. Which one do you want me to delete?"

17. Feature 8 — Explicit Confirmation

High-risk actions require confirmation.

Example:

```
Action:
DELETE_DATABASE

Database:
customer_db

Environment:
production

Risk:
HIGH
```

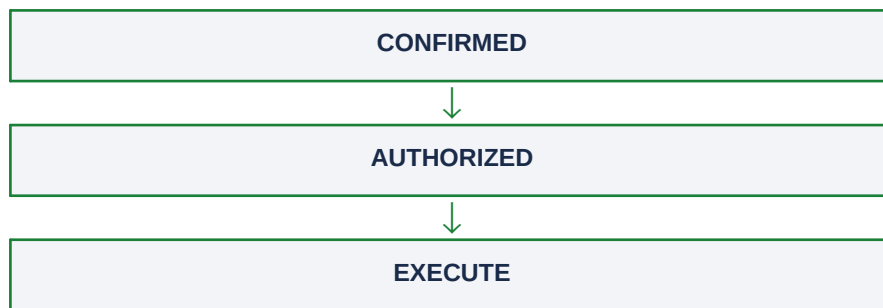
Agent:

"You are requesting permanent deletion of customer_db in production. Do you want me to proceed?"

User:

"Yes."

System:



18. Feature 9 — Confirmation Must Be Specific

Generic conversational context should not automatically count as confirmation.

Example:

Agent:

"You want to delete customer_db from production."

User:

"Okay."

The system should determine whether “Okay” is sufficient confirmation based on the configured confirmation policy.

For high-risk actions, the preferred behavior is explicit confirmation.

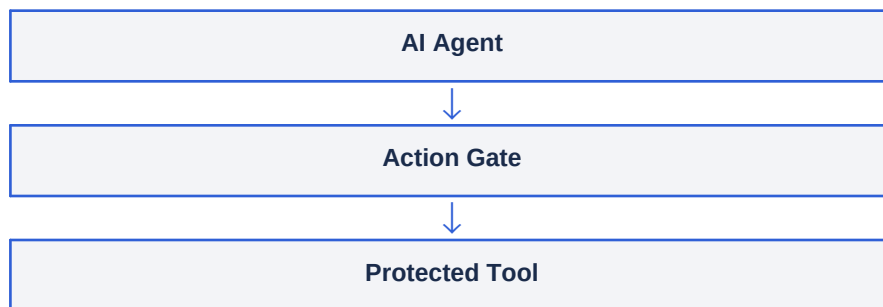
Example:

“Yes, delete it.”

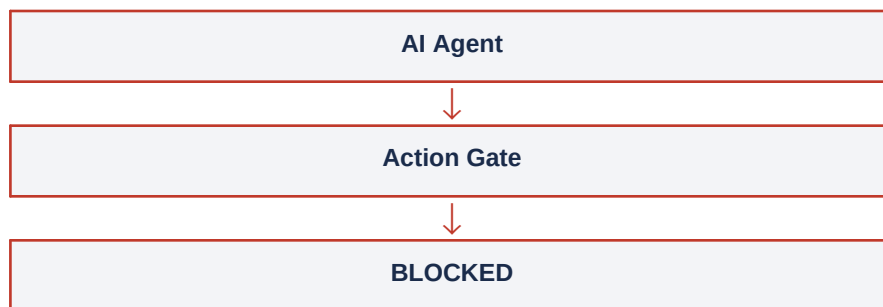
19. Feature 10 — Action Blocking

The Gate must have a hard enforcement point.

The AI cannot directly call the protected tool. Instead:



If validation fails:



The underlying tool must never receive the unauthorized request.

20. Feature 11 — Audit Trail

Every action attempt must be recorded.

Example:

```
{
  "timestamp": "2026-09-06T22:30:14Z",
  "action": "DELETE_DATABASE",
  "parameters": {
    "database": "customer_db",
    "environment": "production"
  },
  "sources": {
    "database": "USER_EXPLICIT",
    "environment": "USER_EXPLICIT"
  },
  "risk": "HIGH",
  "confirmation": true,
  "decision": "AUTHORIZED"
}
```

For blocked requests:

```
{
  "action": "DELETE_DATABASE",
  "decision": "BLOCKED",
  "reason": "Missing explicit environment"
}
```

21. Feature 12 — Visual Security Dashboard

The frontend should display the current action state.

Example:

```
VOICE ACTION GATE
-----
LIVE CONVERSATION

"Deploy version 4.8.2 to production."
-----
DETECTED ACTION
DEPLOY_APPLICATION

Version      4.8.2      OK
Environment  Production  OK
Application  Missing    X

RISK LEVEL      HIGH

STATUS: BLOCKED

Reason: Application not explicitly stated
```

22. Supported MVP Actions

The MVP should contain three major action categories.

Action A — Deploy Application

```
deploy_application(
    application,
    version,
    environment
)
```

Risk:

HIGH

Action B — Delete Database

```
delete_database(
    database,
    environment
)
```

Risk:

HIGH

Action C — Transfer Demo Funds

```
transfer_funds(  
  amount,  
  currency,  
  recipient  
)
```

Risk:

HIGH

This should use **simulated funds only**.

23. Example User Journey

Step 1

User:

"Delete the customer database."

System detects:

```
Database: customer_db ✓  
Environment: missing ✕  
Confirmation: missing ✕
```

Action:

```
BLOCKED
```

Step 2

Agent asks:

"Which environment?"

User:

"Production."

System:

```
Database: customer_db ✓  
Environment: production ✓  
Confirmation: missing ✕
```

Step 3

Agent asks:

"You are requesting permanent deletion of customer_db in production. Do you want me to proceed?"

User:

"Yes."

System:

```
Database: explicit ✓  
Environment: explicit ✓  
Confirmation: explicit ✓
```

```
ACTION GATE
```

```
↓
```

```
AUTHORIZED
```

24. Failure Scenario

User:

"Delete the old production database."

System has:

```
database = unknown  
environment = production
```

Instead of guessing:

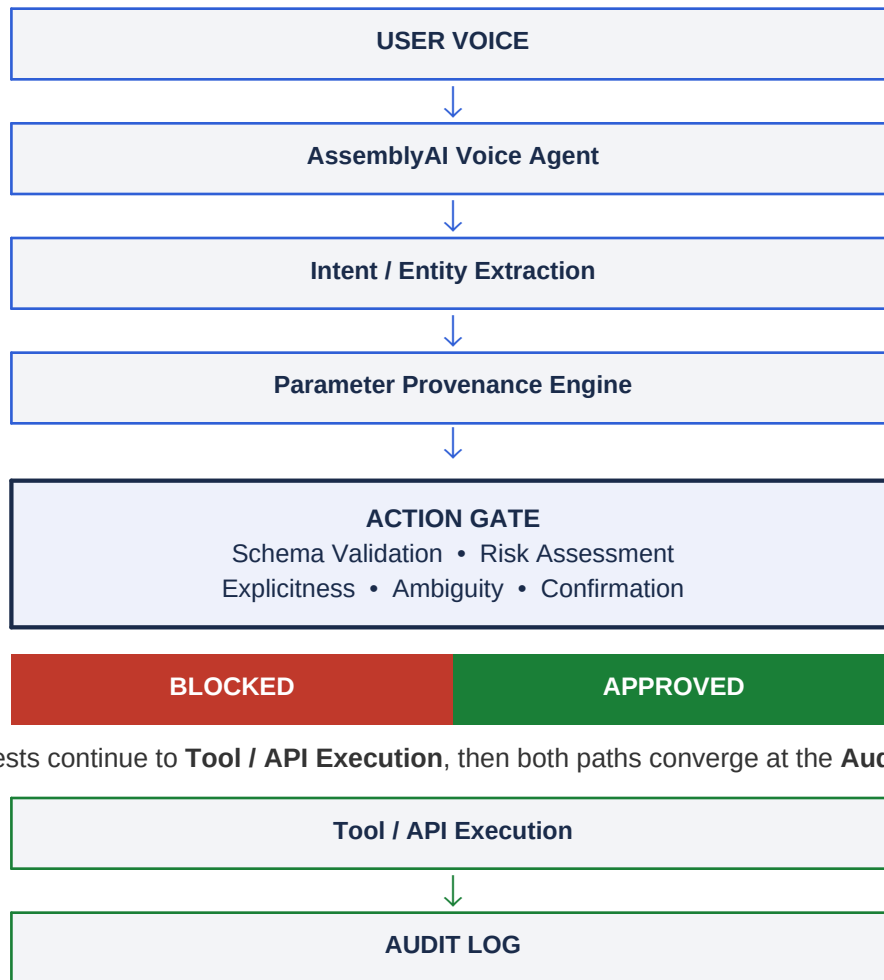
```
BLOCK
```

Agent:

"Which database should I delete?"

This demonstrates the fundamental product behavior.

25. System Architecture



Approved requests continue to **Tool / API Execution**, then both paths converge at the **Audit Log**.

26. Technology Stack

Frontend

- Next.js
- React
- Tailwind CSS
- WebSocket for real-time state updates

Backend

- Python
- FastAPI
- WebSockets

Voice

- AssemblyAI Voice Agent API
- Real-time speech recognition
- Voice responses

AI

- LLM with structured output
- Function/tool calling
- JSON schema validation

Database

- PostgreSQL for production-style architecture
- SQLite acceptable for MVP

Deployment

Potential:

- Vercel — frontend
- Railway / Render — backend
- PostgreSQL hosted database

27. Data Model

Action

```
Action
-----
id
name
description
risk_level
requires_confirmation
created_at
```

ActionParameter

```
ActionParameter
-----
id
action_id
name
type
required
must_be_explicit
```

Execution

```
Execution
-----
id
action_id
status
risk_level
created_at
completed_at
```

ParameterEvidence

```
ParameterEvidence
-----
id
execution_id
parameter_name
value
source
confidence
```

AuditLog

```
AuditLog
-----
id
execution_id
event
reason
timestamp
```

28. Action Gate Decision Engine

Conceptually:

IF action does not exist:
BLOCK

IF required parameter is missing:
BLOCK

IF high-risk parameter is inferred:
BLOCK

IF parameter is ambiguous:
BLOCK

IF high-risk action requires confirmation
AND confirmation is missing:
ASK FOR CONFIRMATION

IF all requirements pass:
AUTHORIZE

EXECUTE TOOL

CREATE AUDIT LOG

29. Security Requirements

SR-1 — AI must never directly execute protected high-risk tools.

SR-2 — All high-risk parameters must have explicit user provenance.

SR-3 — Missing parameters must cause execution to fail closed.

SR-4 — Ambiguous parameters must not be automatically resolved for high-risk actions.

SR-5 — Every authorization decision must be logged.

SR-6 — The system must distinguish between: User statement, System context, AI inference, Default value.

SR-7 — Demo financial and destructive operations must be simulated.

30. Functional Requirements

ID	Requirement	Priority
FR-01	Receive real-time voice input	P0
FR-02	Identify requested action	P0
FR-03	Extract parameters	P0
FR-04	Track parameter provenance	P0
FR-05	Validate action schema	P0
FR-06	Detect missing parameters	P0
FR-07	Detect ambiguity	P0
FR-08	Classify action risk	P0
FR-09	Require confirmation	P0
FR-10	Block unauthorized execution	P0
FR-11	Execute approved tools	P0
FR-12	Generate audit log	P0
FR-13	Display live security state	P1
FR-14	Custom action schemas	P1
FR-15	Historical audit dashboard	P1

31. Non-Functional Requirements

Performance

Target conversational response latency:

< 2 seconds where technically feasible

Reliability

The Gate should fail closed for high-risk actions.

If validation cannot be completed:

DO NOT EXECUTE

Explainability

Every blocked action should provide a human-readable reason.

Example:

"Blocked because the production environment was inferred rather than explicitly stated."

Security

No secrets should be exposed in the UI or logs.

32. Hackathon Demo Script

Demo 1 — AI Guessing

User:

"Delete the old database."

AI identifies multiple possibilities.

Gate:

BLOCKED
Reason:
Ambiguous database

Demo 2 — Missing Authorization

User:

"Delete customer_db."

Gate:

```
Database: explicit ✓  
Environment: missing ✕  
Confirmation: missing ✕
```

Agent asks for environment.

Demo 3 — Inference Attack

User:

"It's probably production."

Gate:

```
Environment:  
PROBABLY PRODUCTION  
  
Source:  
UNCERTAIN  
  
BLOCKED
```

This is the key moment.

Demo 4 — Valid Authorization

User:

"Production."

Agent:

"You want me to permanently delete customer_db in production. Do you confirm?"

User:

"Yes."

Dashboard:

```
ACTION AUTHORIZED  
  
All parameters explicit  
Confirmation verified  
Risk policy satisfied  
  
Executing...
```

33. Success Metrics

For the hackathon MVP:

Security

- 100% of high-risk demo actions pass through the Action Gate.
- 0 unauthorized simulated executions.
- 100% of blocked actions provide a reason.

Voice

- Successful extraction of supported action parameters.
- Natural conversational clarification.
- Successful confirmation workflow.

UX

A first-time user should understand the product within **30 seconds** of seeing the dashboard/demo.

34. Competitive Differentiation

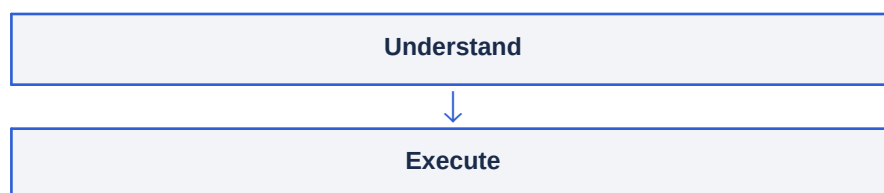
Voice Action Gate is not simply:

“An AI assistant that can execute tools.”

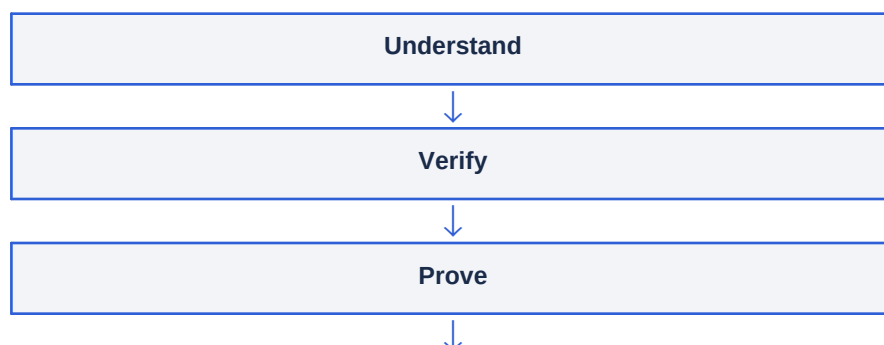
Its key proposition is:

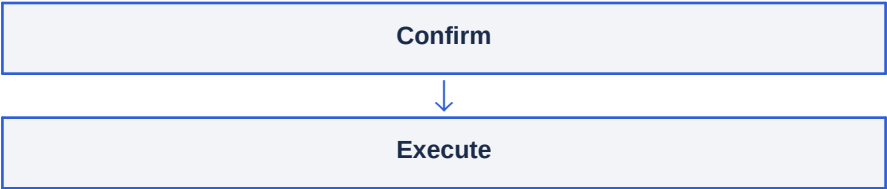
“An authorization firewall for autonomous AI agents.”

Traditional agent:



Voice Action Gate:





The product treats **user speech as evidence**, rather than simply conversational context.

35. Future Roadmap

Phase 1 — Hackathon MVP

- AssemblyAI voice agent
- 3 protected actions
- Action schemas
- Provenance tracking
- Risk levels
- Confirmation
- Audit log
- Dashboard

Phase 2 — Developer Platform

- SDK
- REST API
- Custom action schemas
- Webhooks
- Authentication
- Role-based policies

Phase 3 — Enterprise

- SSO
- RBAC
- Approval chains
- Organization policies
- SIEM integration
- Compliance reports
- Multi-agent authorization

Phase 4 — Universal Agent Security Layer

Support:

- Voice agents
- Chat agents
- Autonomous agents
- Computer-use agents
- API agents

The long-term vision is to become a **policy enforcement layer for AI agents**.

36. Final Product Positioning

Short Description

Voice Action Gate is a real-time authorization layer for AI voice agents that prevents sensitive actions from being executed using inferred or ambiguous user intent.

One-Liner

“AI can understand your intent. It cannot invent your authorization.”

Pitch

“As AI agents move from answering questions to taking real-world actions, the biggest problem isn’t whether AI can execute a task. It’s whether we can trust that the AI is executing exactly what the user authorized. Voice Action Gate solves that problem by verifying every sensitive action before execution—checking explicit parameters, ambiguity, risk, and confirmation, then creating a traceable audit record.”

37. MVP Definition of Done

The MVP is complete when a user can:

- 1 Speak to the AI agent.
- 2 Request a sensitive action.
- 3 Have the system identify the action.
- 4 Extract required parameters.
- 5 Identify which parameters were explicitly spoken.
- 6 Detect missing or ambiguous parameters.
- 7 Ask the user for clarification.
- 8 Prevent unauthorized execution.
- 9 Request explicit confirmation for high-risk actions.
- 10 Execute a simulated tool after authorization.
- 11 Record the complete decision in an audit log.
- 12 See the entire process live on a dashboard.

Core success condition:

If the AI doesn't have explicit authorization, the action doesn't happen.